DAT535 - Temperature Forecasting

Temperature Forecasting with Distributed Computing and Deep Learning Models

Sigurd Heggemoen
Alan Lici
University of Stavanger, Norway
sheggemoen@gmail.com
alan-lici@hotmail.com

ABSTRACT

This project addresses the challenge of temperature forecasting by developing a scalable and efficient data processing pipeline integrated with machine learning models. Leveraging the Medallion Architecture, the pipeline organizes data into bronze, silver, and gold layers, enabling structured ingestion, pre-processing, and analysis of weather data. Apache Spark is employed for distributed processing, ensuring efficient handling of large-scale datasets and integration with Hadoop's Distributed File System (HDFS).

The forecasting system incorporates three machine learning models—linear regression, Long Short-Term Memory (LSTM), and Convolutional Neural Networks (CNN). Linear regression provides a computationally lightweight baseline, while LSTM captures long-term temporal dependencies, and CNN focuses on localized patterns in the data. Among these, the LSTM model consistently achieved the best performance, with the lowest Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) for shorter prediction horizons, highlighting its effectiveness for capturing complex temporal dependencies.

To improve performance, Spark configurations and machine learning hyperparameters were optimized. The results demonstrate the proposed system's effectiveness in delivering accurate temperature forecasts while maintaining scalability and efficiency. These findings make the system a valuable tool for agriculture, energy management, and public safety applications.

KEYWORDS

Temperature forecasting, Distributed computing, Apache Spark, Machine learning models, Medallion architecture

1 INTRODUCTION

The demand for accurate weather forecasting has been high across various sectors, including agriculture, energy management, transportation, and public safety. Weather patterns are complex and often involve a combination of seasonal trends and unpredictable changes. Accurate forecasting helps optimize decision-making, improve resource allocation, and ensure safety in diverse applications. However, predicting future weather conditions, particularly temperature, remains challenging due to time-series data variability and complex dependencies. Precise weather forecasts are very relevant for several agencies in society and affect several sectors, such as agriculture, energy management, transport and public safety. [8]

Supervised by Tomasz Wiktorski and Jayachander Surbiryala.

Project in Data-intensive Systems and Algorithms (DAT535), IDE, UiS 2024.

This project addresses the challenge of temperature prediction by using data processing and machine learning techniques. The focus of this project is on developing an end-to-end data processing pipeline using Apache Spark to enable the forecasting of temperature values. The pipeline follows the Medallion Architecture, which provides a clear structure for organizing and transforming the data through distinct layers. Additionally, we explore multiple machine learning models, including linear regression, Long Short-Term Memory (LSTM), and Convolutional Neural Network (CNN), to compare and assess their effectiveness in forecasting temperature trends.

The core research problem we address in this project is how to effectively predict daily temperatures using a distributed data processing pipeline and machine learning models. The challenge lies in handling large-scale unstructured data, transforming it to derive meaningful features, and then applying models capable of capturing both linear and non-linear relationships. Given the importance of accurate temperature prediction for many industries, our goal is to build a scalable, efficient, and accurate forecasting solution that provides insights into future temperature patterns.

2 BACKGROUND

This project focuses on building a data processing pipeline using Hadoop and Apache Spark for distributed computing. The pipeline follows the Medallion Architecture and uses machine learning models such as linear regression, LSTM, and CNN. The performance of the different machine learning models is calculated using different performance metrics.

2.1 Distributed Computing with Hadoop

Apache Hadoop is a popular framework that enables distributed storage and computing across clusters of computers. It is designed to scale from a single server to thousands of machines, with each one offering local computation and storage. Hadoop consists of Hadoop Distributed File System (HDFS) and Yet Another Resource Negotiator (YARN). HDFS provides a shared file system across computers for distributed storage, and YARN allocates system resources for processes.[1]

2.2 Apache Spark

Apache Spark is an engine built for large-scale data processing. It is based on Hadoop MapReduce but extends it with in-memory processing, resulting in significantly faster execution time. Spark uses Resilient Distributed Datasets (RDD), which are stored in cached memory, limiting I/O from disk. Additionally, Spark provides advanced libraries like MLlIb and Spark TensorFlow Distributor for

machine learning, which leverage this in-memory storage to speed up processing. [1]

2.3 Medallion Architecture

The Medallion Architecture is a data pipeline design framework that organizes data processing into distinct layers. It consists of three layers: bronze, silver, and gold. The bronze layer handles the ingestion of raw data into the pipeline. The silver layer refines and structures this raw data, performing essential pre-processing tasks to prepare it for further analysis in the next stage. The gold layer is the final stage, where analytics, reporting, and machine learning are done. [9]

2.4 Machine Learning Models for Forecasting

Time series forecasting is the process of predicting the future values based on patterns observed from historical data. Several different machine-learning models have been used, and in this project, we employ three different models proven to be effective: linear regression, LSTM and CNN.

2.4.1 Linear Regression. Linear regression is widely used in machine learning because of its simplicity and effectiveness. It works by identifying a linear relationship between the input features and the target variable. The equation, as shown in Equation 1, predicts the target variable y as a weighted sum of the input features X_1, X_2, and a bias term B_0 . The goal is to determine the coefficients B_1, B_2 , and B_0 that minimize the sum of squared differences between the predicted values and the actual outcomes. [14]

$$y = B_0 + B_1 * X_1 + B_2 * X_2 \tag{1}$$

2.4.2 Long Short-Term Memory. LSTM is a Recurrent Neural Network (RNN) type created to solve the "long-term dependency problem". In standard RNNs, important information often gets lost as new data is processed, making it challenging to capture long-term patterns. LSTMs mitigate this by adding a memory cell mechanism, which helps the network preserve crucial information over extended sequences. [16]

LSTMs are well-suited for time series forecasting because they are capable of capturing patterns and dependencies in the data. These models have been used in areas like stock price prediction and weather forecasting. [19]

2.4.3 Convolutional Neural Networks. CNN is a type of deep-learning neural network widely used for image classification tasks. They excel at detecting patterns in images through convolution layers, which apply filters to detect patterns. However, CNNs have also been proven effective in time series forecasting by identifying patterns over time. One of its key strengths is its ability to detect patterns within local regions, allowing it to capture short-term features. Moreover, by stacking multiple convolutional layers, CNNs can learn both short-term and long-term patterns effectively. Moreover, CNNs are processing the time series in parallel, making it more efficient compared to LSTM, which processes the time series sequentially. [15]

2.5 Performance Metrics

2.5.1 Mean Absolute Error. Mean Absolute Error (MAE) is a popular metric for evaluating regression models. It takes the mean of the absolute difference between the predicted and actual values, as shown in equation (2). [11]

$$\text{Mean Absolute Error} = \frac{1}{N}\sum_{i=1}^{N}|P_i - Y_i| \tag{2}$$

2.5.2 Mean Squared Error. Mean Squared Error (MSE) is the mean of the squared distance between the predicted and the actual values, as shown in equation (3). [5]

$$\text{Mean Squared Error} = \frac{1}{N}\sum_{i=1}^{N}(P_i - Y_i)^2 \tag{3}$$

2.5.3 Root Mean Squared Error . Root Mean Squared Error (RMSE) is the square root of the MSE, as shown in equation (4). [6]

$$\text{Root Mean Squared Error} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(P_i - Y_i)^2} \tag{4}$$

2.5.4 Accuracy. The accuracy is the accurately predicted values divided by the total number of predictions.

3 METHOD

In this chapter, we provide a detailed explanation of our approach to solving the temperature prediction problem, focusing on the design, analysis, and optimization aspects of our data processing pipeline and machine learning models.

3.1 Design

The design of our temperature forecasting system follows a structured approach using the Medallion Architecture. This architecture organizes data processing into three distinct layers as shown in Figure**1**:

- **Bronze Layer**: Weather data from the Frost API is ingested and unstructured to increase the complexity of the processing tasks. The data includes temperature readings with metadata about measurement conditions and timestamps.
- **Silver Layer**: In this layer, we clean and pre-process the data, transforming it into a structured format suitable for machine learning. This involves parsing the raw logs, extracting temperature values, and engineering features such as sine and cosine transformations to capture seasonality. The processed data is then stored as a DataFrame in Parquet in HDFS for efficient storage and retrieval.
- **Gold Layer**: Finally, we use the cleaned and transformed data to train multiple machine learning models, including linear regression, LSTM, and CNNs, to forecast future temperature values.

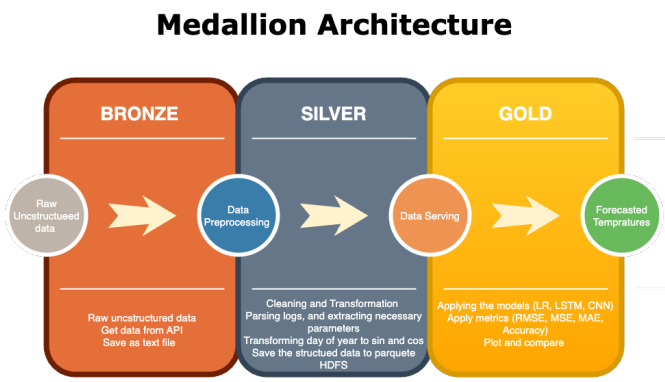

Figure 1: This figure illustrates the Medallion Architecture used in the project, with three layers; bronze, silver and gold

The pipeline was designed to use Apache Spark for distributed data processing, ensuring scalability and efficiency. Unique features of our design include the use of feature engineering to capture seasonal patterns and the combination of multiple machine-learning models to compare their effectiveness.

3.1.1 Database creation. In this part of our project, we focus on data ingestion and the creation of the initial database. The raw weather data is retrieved from the Frost API, which provides detailed weather observations in a semi-structured JSON format. [4] We further unstructured this data to increase the complexity, simulating real-world unstructured log data that requires additional pre-processing.

The ingested data is stored as text files in HDFS to serve as the foundational data source for the rest of the pipeline. Each record in the dataset contains temperature readings along with metadata such as timestamps, measurement conditions, and other related information. This ensures that we capture all necessary raw data for further processing and transformation steps.

The Bronze Layer provides a robust foundation for storing raw, unprocessed weather data, ready for the next stages of the data pipeline.

3.1.2 Data Pre-Processing. In this part we clean and pre-process the ingested data, transforming it from an unstructured log format into a structured form suitable for machine learning models. The first step involves reading the raw data stored in HDFS and parsing each log entry to extract meaningful features.

We process each record to extract the date, temperature value, and additional features to represent seasonality, such as sine and cosine transformations of the day of the year. This feature engineering step helps our models to capture the seasonal trends in weather data given by the equations (5). [10]

$$x_{sin} = \sin\left(\frac{2 * \pi * DayOfYear}{365}\right), \quad x_{cos} = \cos\left(\frac{2 * \pi * DayOfYear}{365}\right) \tag{5}$$

Once the raw data is parsed, we aggregate it to compute the daily average temperature, which serves as the basis for our next analysis and modelling tasks.

After pre-processing, the cleaned data is saved in HDFS as a DataFrame in Parquet format. This format ensures efficient storage and retrieval, facilitating quick access during the modelling stage. [17] The Silver layer provides a structured dataset to be used in training machine learning models.

3.1.3 Linear Regression. Linear regression is used as a baseline model in this project to establish a reference point for evaluating the performance of more complex models, such as LSTM and CNN.

By using the sine and cosine transformations of the day of the year as input features, linear regression captures the seasonal patterns in temperature data allowing the model to predict future temperatures based on observed trends. As a computationally lightweight model, linear regression provides a fast and efficient approach to forecasting and a good baseline model for comparing more advanced machine learning models. [14]

3.1.4 LSTM. The LSTM model is designed to learn and capture temporal dependencies within time series data, making it suitable for sequential data like temperature readings. LSTM is effective for capturing long-term dependencies, which is key for forecasting temperature. [16]

Our LSTM model uses the input as a sequence of temperature values combined with sine and cosine features to represent seasonal patterns. This design enables the model to use both temporal and cyclical information, which are key characteristics of weather data.

A stacked architecture with multiple LSTM layers was chosen to enhance the model's capacity to learn complex patterns from the data. This architecture is designed to allow the model to learn long-term dependencies in the data. [19] By integrating the LSTM model with distributed training capabilities, the design also ensures scalability, allowing efficient handling of large datasets in a distributed computing environment.

3.1.5 CNN. The CNN is included in this project due to its effectiveness in detecting localized patterns within time-series data, allowing it to capture and identify patterns in temperature trends. [15]

The model processes input sequences, including historical temperatures and sine-cosine seasonal features, using convolutional layers to extract meaningful local patterns. Pooling layers are applied to reduce dimensionality, and dense layers aggregate the extracted features to predict future temperature values. By using parallel processing, CNNs offer computational efficiency and scalability, particularly in distributed environments.

As part of the overall design, CNNs balance the LSTM's focus on long-term dependencies by emphasizing short-term trends, enabling a comprehensive approach to temperature forecasting.

3.1.6 Model Evaluation. To ensure a fair comparison of model performance, all models are evaluated using the same performance metrics, specifically MAE, MSE, RMSE, and accuracy. Since accuracy is not typically informative for continuous variables, we define it here with a margin of error of one degree. Therefore a prediction is considered correct if it falls within one degree of the true value.

3.2 Analysis

The performance evaluation of our temperature prediction system focuses on the chosen design's effectiveness and ability to meet the project's scalability, flexibility, and accuracy goals. The Medallion Architecture plays a central role in achieving these goals by providing a structured and modular approach to data processing. By dividing the data into bronze, silver, and gold layers, the architecture ensures that transformations occur in stages, facilitating flexibility and independent optimization at each layer. This modular structure also simplifies debugging and improvements to individual pipeline components, enhancing overall system efficiency.

The inclusion of three machine learning models—linear regression, LSTM, and CNN—ensures a comprehensive approach to temperature prediction. Linear regression serves as a baseline model, offering simplicity and a reference point for comparison. LSTM is well-suited for capturing long-term dependencies in sequential weather data, while CNN complements this by focusing on localized patterns, providing a balance between short- and long-term forecasting capabilities. [16] [15] By combining these models we use their individual strengths, addressing the diverse relationships in temperature data.

Apache Spark's distributed processing capabilities are important to handling large-scale weather data efficiently. Spark's ability to parallelize computations and perform in-memory processing minimizes latency during data transformation and model training. Its compatibility with Hadoop and HDFS further enhances the integration between data storage and processing, contributing to a scalable and efficient system. [1]

To simplify the training and evaluation process, a single model per algorithm was trained to forecast for multiple horizons (1, 7, and 30 days). While training separate models for each horizon could improve accuracy, this approach ensures flexibility and reduces complexity, making optimization and debugging more manageable. This design trade-off balances accuracy with practical considerations for scalability and maintainability.

Overall, the system demonstrates the effectiveness of a distributed architecture and the complementary strengths of multiple machine learning models, delivering an accurate and scalable solution for temperature forecasting.

3.3 Optimization

The design of our temperature forecasting pipeline follows a structured approach, utilizing the Medallion Architecture to divide processing tasks across different layers. This setup allows us to tune and optimize each layer individually, targeting specific processing and resource requirements in the silver and gold layers.

The silver layer handles data pre-processing, specifically converting hourly temperature readings into average daily temperatures. Since pre-processing is computation-heavy and benefits from parallelized tasks, we planned to experiment with different Spark algorithms to achieve optimal performance.

The gold layer is designed to handle temperature forecasting with machine learning models. This layer includes models of varying complexity: linear regression, LSTM, and CNN. While linear regression is computationally lightweight and does not require extensive optimization, the more complex LSTM and CNN models demand specific tuning. Therefore, we planned to experiment with different batch sizes and learning rates to test and identify configurations that could optimize training efficiency and accuracy.

Resource allocation within the Spark environment was carefully structured to optimize the pipeline. For this, we configured the Spark environment to explore varying levels of parallelism by adjusting the number of cores per task and the number of executors. High core allocations per task support CPU-intensive processes, while low allocations allow for more parallelism, benefiting workloads with many tasks. We also experimented with the number of executors, where many executors are good for processes that can be effectively partitioned and processed on multiple executors. [18] Additionally, the optimization plan included scaling up the dataset to emulate real-world conditions to test the scalability and efficiency of the system.

Throughout the optimization process, the Spark History Server was used to monitor job performance to identify bottlenecks and facilitate configuration adjustments. This systematic approach allowed us to evaluate which changes had the greatest positive impact on our pipeline.

3.4 Implementation

This section presents our approach for implementing a time series forecasting pipeline using cloud resources and machine learning models. We outline the cloud setup, pre-processing techniques, and the details of the machine learning models.

3.4.1 Cloud Setup. The cloud infrastructure for this project is deployed on OpenStack. It consists of a master node and three data nodes, all running Ubuntu 20.04 with Hadoop and Apache Spark. The nodes are configured with four CPU cores and eight GB of RAM each. We utilized HDFS as the primary storage solution due to its scalability and ability to manage large datasets across nodes. This setup enhances performance by enabling distributed computations and optimizing resource usage across the data nodes. To optimize the performance of the Spark environment, we enabled Spark to use Hadoop YARN, which provides improved resource management and scheduling. [1]

An initial challenge was that not all three data nodes were utilized effectively, as data distribution was uneven. To address this, we increased the HDFS replication factor from 2 to 3, ensuring each data node maintained a copy of the data. This adjustment not only improved fault tolerance but also enabled balanced workload distribution across the cluster.

3.4.2 Database Creation. Historical temperature data was obtained from the Frost API, which provides access to the archive of historical climate and weather data from the Norwegian Meteorological Institute (Met Norway). [4] To enable access, we registered as users on the Frost platform (frost.met.no) and obtained a unique client ID and client secret for secure API authentication.

We extracted temperature data spanning from January 1, 2000, to October 18, 2024. Given that Frost API requires the specification of weather stations for data retrieval, we evaluated several stations based on data completeness and chose Station SN19710, located in Asker, due to its minimal missing data. This returned 121,414

hourly temperature readings in JSON format, a volume sufficient to support our intended forecasting objectives.

The JSON data was then processed into an unstructured text format and stored in HDFS, creating a file of approximately 11.5 MB. This transformation to an unstructured format aligns with the project's data processing requirements and facilitates efficient storage and retrieval within the HDFS framework.

One challenge we encountered was that the dataset was too small to allow effective monitoring and observation for optimization purposes. To address this, we duplicated each entry 150 times, increasing the dataset size from 11.5 MB to 1.7 GB. Thus allowing for a more realistic simulation of large-scale data and providing a clearer basis for evaluating the performance of different optimization techniques.

3.4.3 Data Pre-Processing. The data pre-processing phase involved transforming the ingested raw temperature data from an unstructured format into a structured form suitable for machine learning tasks. For this we used Apache Spark's capabilities for distributed data processing to efficiently clean, transform, and aggregate the data across multiple nodes.

The initial step involved reading the raw data stored in HDFS and transforming it using Spark RDD transformations. The data was loaded as an RDD, and we used the .map() function to parse each line and extract meaningful information, such as the date and temperature values and additional features that capture seasonality.

One significant challenge was capturing the seasonal patterns inherent in temperature data. Weather data has strong seasonal trends, which can be difficult for machine learning models to learn without explicit feature engineering. To address this, we applied sine and cosine transformations to the "day of the year" feature to effectively represent the cyclical nature of temperature variations. These transformations helped the models to capture temporal relationships better, and the implementation for these transformations is shown in the Code 1.

```
day_of_year_sin = round(math.sin(2 * math.pi *
    day_of_year / 365), 2)
day_of_year_cos = round(math.cos(2 * math.pi *
    day_of_year / 365), 2)
```

Listing 1: Day of Year Sin/Cos Transformation

Another challenge we faced was aggregating hourly temperature values into daily averages. For this, we used Spark's groupByKey() and reduceByKey() functions. The groupByKey() function allowed us to group temperature values by their respective dates, enabling us to perform daily aggregation. However, to improve performance and reduce shuffle overhead, we also experimented with reduceByKey(), which directly aggregates values based on a specified key. This approach is more memory-efficient compared to groupByKey(), particularly for large datasets, as it reduces the amount of data that needs to be shuffled between nodes. [18]

Once the data was aggregated, it was converted into a DataFrame. We used the Parquet file format for storing the cleaned data in HDFS, as it provides efficient storage and fast retrieval capabilities, which are crucial for the next model training stages. [17]

3.4.4 Linear Regression. In the gold layer, we implemented the linear regression model to establish a baseline prediction for temperature forecasting. This was used to evaluate the potential of simpler models compared to more complex ones like LSTM and CNN.

We started by using the cleaned and preprocessed data, stored in HDFS as a Parquet file. The data was loaded into a Spark DataFrame, after which it was split into training and testing sets, with 80% of the data used for training and the remaining 20% for testing. The split was done chronologically to preserve the temporal dependency within the dataset.

The features used for training were the sine and cosine components derived from the day of the year, which represent seasonality in temperature variations. To train the linear regression model, we used the VectorAssembler class from PySpark to combine these features into a single vector column. [12]

The linear regression model was created and trained using PySpark's LinearRegression class from the MLlib library. [12] The model was configured with the label column as "temperature" and the "features" column containing the feature vector. It was fit on the training data to derive coefficients and intercepts that best minimized the error between predicted and actual temperatures.

3.4.5 Long Short-Term Memory. We implemented an LSTM model to predict temperature values over various forecast horizons. Specifically, we chose to use a unified LSTM model for predicting temperature across 1-day, 7-day, and 30-day horizons rather than training separate models for each forecasting interval. While developing individual models for each horizon could give better predictive accuracy, our approach aimed to create a more generalized model that would be easier to optimize, troubleshoot, and deploy in a broader context.

The cleaned temperature data, stored in HDFS in Parquet format, was loaded into a Spark DataFrame. The dataset was split into training, validation, and testing, using a ratio of 60% for training, 20% for validation, and 20% for testing.

We used TensorFlow to define the model and Spark TensorFlow Distributor with MirroredStrategy to distribute training across three nodes. [7] [3] We created input sequences from the data where each sequence comprised 90 days of historical data, with temperature, sine, and cosine transformations of the day of the year used as features. The target output consisted of temperature predictions for the next 30 days. This setup allowed the model to use past data effectively to learn patterns necessary for forecasting future temperature trends.

The LSTM model consisted of a sequential architecture with two stacked LSTM layers, each containing 50 units. The historical sequences (90 days) were used as input to the LSTM layers, capturing both linear and seasonal trends through temperature, sine, and cosine features. The final dense layer of the model produced the predictions for the next 30 days. [19] The Code 2 shows the architecture used for the LSTM implementation.

```
model = tf.keras.Sequential([
    tf.keras.layers.LSTM(50, return_sequences=True,
        input_shape=(90, 3)),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.LSTM(50),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(30)
])
```

Listing 2: LSTM Model Architecture

For model optimization, we utilized the MSE as the loss function with a learning rate of 0.001. The model training was initially conducted using a batch size of 64 and 200 epochs. Additionally, we used an early stopping callback, which monitored the validation loss and stopped training when the loss did not improve for 50 consecutive epochs and retrieved the weights when the validation loss was the lowest. This way the best model is evaluated against the test set.

3.4.6 Convolutional Neural Network. The CNN used TensorFlow to define the model and Spark TensorFlow distributor with MirroredStrategy to distribute training across three nodes. [7] [3] The CNN processes the same preprocessed datasets used in the LSTM model, as described in chapter 3.4.5, ensuring consistency across models.

The model is structured with a 1D convolutional layer using 64 filters and a kernel size of 3, followed by a max-pooling layer with a pool size of 2 to reduce dimensionality. The flattened output from the pooling layer is passed to a dense layer with 50 units, followed by a dropout layer with a 20% rate to prevent over-fitting. The final layer consists of 30 units, outputting the predicted temperatures for the next 30 days. The model is compiled using a learning rate of 0.001 and MSE as the loss function. [15] The Code 3 shows the CNN architecture used for the CNN implementation.

```
model = tf.keras.Sequential([
    tf.keras.layers.Conv1D(filters=64, kernel_size=3,
        activation='relu', input_shape=(90, 3)),
    tf.keras.layers.MaxPooling1D(pool_size=2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(50, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(30)
])
```

Listing 3: CNN Model Architecture

Similar to the LSTM, an initial batch size of 64 and an early stopping callback was applied to monitor validation loss. The training process iterates for a maximum of 200 epochs, though it terminates early if no improvement in validation loss is observed for 50 consecutive epochs.

3.4.7 Model Evaluation. After training the models, we evaluated their performance on the test dataset using performance metrics like MAE, MSE, and RMSE. These metrics were calculated using the sklearn.metrics library. We also measured the proportion of predictions within a predefined one-degree margin of error, offering additional insights into the accuracy of the forecasts.

Visualization of model performance was done using the Matplotlib library, which allowed us to create line plots comparing true values against predictions together with a 95% confidence interval.

3.4.8 Optimization . The optimization of the temperature forecasting pipeline is carried out within the Medallion Architecture, which allows for the independent optimization of each processing layer. Specifically, the silver layer handles data pre-processing, while the gold layer is responsible for temperature forecasting using machine learning models. As previously outlined in the design section, the optimization efforts were categorized into algorithm tuning and Spark configuration tuning.

In the silver layer, the focus is on converting hourly temperature data to daily averages. We implemented two distinct algorithms using Spark's distributed computing capabilities. The first algorithm utilizes Spark's groupByKey() function to group temperature data by date and calculates the daily average while also including the sine and cosine values, as shown in the Code 4. The second algorithm, which uses Spark's reduceByKey() function, provides efficient aggregation by reducing data across partitions, as shown in Code 5. This can improve performance when dealing with large datasets.

```
daily_avg_rdd = processed_rdd.groupByKey() \
    .mapValues(lambda temps: list(temps)) \
    .mapValues(lambda temps: (round(statistics.mean([t[0]
        for t in temps])), temps[0][1], temps[0][2]))
```

Listing 4: Daily Average Calculation Using groupByKey

```
daily_avg_rdd = processed_rdd \
    .map(lambda t: (t[0], (t[1][0], t[1][1], t[1][2], 1))
        ) \
    .reduceByKey(lambda a, b: (a[0] + b[0], a[1], a[2], a
        [3] + b[3])) \
    .mapValues(lambda v: (round(v[0] / v[3]), v[1], v[2])
        )
```

Listing 5: Daily Average Calculation Using reduceByKey

For the gold layer, we focused on optimizing the more complex models LSTM and CNN. While linear regression did not require additional tuning due to its simplicity, the LSTM and CNN models were adjusted with different batch sizes of 16 and 64 to assess their impact on training efficiency and model performance. Additionally, we tested two learning rates, 0.001 and 0.01, to identify the rate that gives the best trade-off between training speed and model accuracy.

The optimization of Spark configurations involved testing various setups to adjust the parallelism and resource allocation, specifically targeting the number of executors and cores per task. We experimented with four setups, each adjusting the number of cores per task and the number of executors:

(1) **Maximum core per task:** A setup where three executors used the maximum number of cores (four) but ran only one task at a time. This setup is suitable for CPU-intensive tasks that require dedicated processing resources.
(2) **Maximum parallelism per node:** A setup where three executors used the maximum number of cores (four) and each task was limited to a single core, allowing for higher parallelism. This configuration is beneficial when tasks can be independently processed but do not require extensive resources.
(3) **Maximum executors:** A configuration with ten executors, each using one core. Although we should have been able to have twelve executors with one core each we were limited

by available memory. However, this setup helped explore the trade-offs of scaling the number of executors versus task distribution

(4) **Balanced parallelism:** A configuration where two cores were allocated per task, allowing for three executors running two tasks in parallel, offering a balance between resource utilization and task execution.

However, when applying these configurations to more complex models using Spark's TensorFlow distributor (MirroredStrategy), we faced significant challenges related to task synchronization. MirroredStrategy relies on Spark's barrier execution mode for coordinated parallelism, which requires each executor to have at least one available core for barrier task synchronization in addition to the cores allocated for training. To meet this requirement, we adjusted our configurations to reserve one core per executor for synchronization, ensuring stable execution and preventing deadlocks during distributed training. [2]

(1) **Configuration 1:** Three executors with four cores per executor and two cores per task. This allowed for three parallel jobs, as two cores per executor were reserved for synchronization.
(2) **Configuration 2:** Three executors with four cores per executor and one core per task. This configuration enabled the maximum number of nine parallel jobs by dedicating one core per executor to barrier synchronization.
(3) **Configuration 3:** Six executors with two cores each and one core per task. This setup resulted in six parallel jobs during training, which helped achieve better parallelization while respecting barrier synchronization requirements.

This systematic approach to optimization allowed us to balance parallelism, resource usage, and task synchronization effectively, ultimately enhancing the performance and scalability of our model training pipeline.

4 EXPERIMENTAL EVALUATION

This section evaluates our temperature forecasting pipeline, covering our experimental setup and analyzing results for various model configurations and optimizations. The evaluation assesses model performance across different forecast horizons and the effect of Spark configuration on computational efficiency.

4.1 Experimental setup

The experiments were conducted on an OpenStack cloud environment with one master node and three data nodes, all running Ubuntu 20.04. The nodes are configured with four CPU cores and eight GB of RAM each. We utilized HDFS as the primary data storage, setting the replication factor to three so all nodes maintained a copy of the data. Spark was configured to work with HDFS, enabling parallelized data processing and model training. Spark was integrated with Hadoop YARN to maximize resource allocation efficiency, which provided dynamic scheduling and enhanced resource management across nodes. [1]

All configurations and code used in this project are available on GitHub. [13]

4.2 Results

4.2.1 Spark Performance Optimization Results. In this section, we evaluate the impact of various Spark configurations on preprocessing, linear regression, and deep learning model training times. Different configurations were tested to optimize resource allocation, focusing on balancing parallelism, resource usage, and task synchronization. Results are presented in the following sections.

Data Pre-Processing and Linear Regression Results. Four configurations were evaluated for the data pre-processing and linear regression stages, as detailed in chapter 3.4.8. The configurations focused on different aspects of parallelism and resource allocation to optimize execution time. The `groupByKey` algorithm was used for data pre-processing to calculate daily temperature averages. Each configuration was tested across five runs, with the average execution time and standard deviation calculated for consistency. The results for pre-processing and linear regression are summarized in Table 1.

Pre-Processing		
Configuration	**Execution Time (min)**	**Standard Deviation (min)**
1	2.15	0.05
2	1.09	0.03
3	1.4	0
4	1.38	0.04
Linear Regression		
Configuration	**Execution Time (min)**	**Standard Deviation (min)**
1	0.76	0.01
2	0.76	0.02
3	1.28	0.04
4	0.79	0.02

Table 1: Execution time and standard deviation for preprocessing and linear regression, averaged over five runs under different configurations.

CNN and LSTM Model Training Results. Different configurations were required for the CNN and LSTM models to address the specific needs of distributed TensorFlow tasks, as discussed in chapter 3.4.8. Each configuration was tested across five runs, with the average execution time and standard deviation calculated for consistency. The results for LSTM and CNN are summarized in Table 2.

LSTM		
Configuration	**Execution Time (min)**	**Standard Deviation (min)**
1	11.60	0.80
2	21.20	2.71
3	15.40	1.50
CNN		
Configuration	**Execution Time (min)**	**Standard Deviation (min)**
1	5.54	1.36
2	10.16	1.16
3	10.96	2.56

Table 2: Execution time and standard deviation for LSTM and CNN, averaged over five runs under different configurations.

Data Preprocessing Algorithm Comparison. In this section, we present the execution times for the two different algorithms used in the data pre-processing stage: the groupByKey() and reduceByKey() methods for calculating daily temperature averages, as discussed in chapter 3.4.8. Both algorithms were tested under the best-performing Spark configuration, Configuration 2 - Maximum Parallelism per Node. Each algorithm was executed across five runs, and the average execution time and standard deviation were calculated for consistency. The results are summarized in Table 3.

Algorithm	Execution Time (min)	Standard Deviation (min)
GroupByKey	0.54	0.02
ReduceByKey	0.53	0.01

Table 3: Execution time and standard deviation for data preprocessing algorithms, averaged over five runs, using the best Spark configuration (Configuration 2 -maximum parallelism per node).

4.2.2 Model Performance and Hyperparameter Tuning Results. In this section, we present the performance of the linear regression, CNN, and LSTM models, including the impact of hyperparameter tuning for the CNN and LSTM models under the best-performing Spark configuration (Configuration 1 - Maximum Core per Task). The hyperparameters tuned include batch size and learning rate, as discussed in Chapter 3.4.8. For each model, we evaluated performance using the metrics MAE, MSE, RMSE and accuracy with a margin of error of 1 degree. Additionally, we included figures in Appendix 7 that display the actual values versus the predicted values, along with a 95% confidence interval, for the best configuration of each model.

We also include the execution time for each tuning combination for CNN and LSTM models, demonstrating the impact of hyperparameter choices on performance and efficiency.

Linear Regression Results. The linear regression model was not tuned, and the forecast horizon is independent of the previous temperatures, so only one result is presented, as discussed in Chapter 3.4.4. The performance metrics are summarized in Table 4.

MAE	MSE	RMSE	Accuracy (1° Error)
2.66	11.41	3.38	0.23

Table 4: Linear Regression Performance Metrics.

LSTM and CNN Results. For the LSTM and CNN models, different batch sizes and learning rates were evaluated to optimize their performance. Additionally, we evaluated the model's performance to forecast one, seven and 30 days ahead, where results, including execution time, are shown in Table 5.

Model	Learning Rate	Batch Size	MAE	MSE	RMSE	Accuracy	Time (min)
1 Day Forecast							
LSTM	0.001	64	2.28	8.26	2.88	0.27	11.60
CNN	0.001	64	2.26	7.76	2.79	0.26	5.54
LSTM	0.001	16	2.21	7.68	2.78	0.28	29.00
CNN	0.001	16	2.46	9.43	3.07	0.26	15.00
LSTM	0.01	64	1.95	5.96	2.44	0.32	10.00
CNN	0.01	64	3.33	18.41	4.29	0.19	5.10
7 Day Forecast							
LSTM	0.001	64	2.63	11.22	3.35	0.25	11.60
CNN	0.001	64	2.67	11.31	3.36	0.24	5.54
LSTM	0.001	16	2.68	11.56	3.40	0.23	29.00
CNN	0.001	16	2.66	11.38	3.37	0.25	15.00
LSTM	0.01	64	2.68	11.63	3.41	0.24	10.00
CNN	0.01	64	3.14	17.19	4.15	0.22	5.10
30 Day Forecast							
LSTM	0.001	64	2.68	11.83	3.44	0.25	11.60
CNN	0.001	64	2.74	12.38	3.52	0.24	5.54
LSTM	0.001	16	2.76	12.46	3.53	0.24	29.00
CNN	0.001	16	2.71	12.33	3.51	0.24	15.00
LSTM	0.01	64	2.69	11.65	3.41	0.24	10.00
CNN	0.01	64	3.36	20.18	4.49	0.20	5.10

Table 5: Model performance metrics for LSTM and CNN for different forecast periods (1 day, 7 days, and 30 days), evaluated under the best Spark configuration (Configuration 1 - maximum core per task).

4.3 Discussion

4.3.1 Spark Performance Optimization Discussion.

Data Pre-Processing and Linear Regression. Table 1 demonstrates that Configuration 2 ("Maximum Parallelism per Node") yields the best execution times for data pre-processing, with a processing time of 1.09 minutes. This configuration maximizes task parallelism by allocating one core per task, allowing multiple tasks to execute simultaneously. However, for linear regression, Configurations 1 and 2 show similar execution times (0.76 minutes), indicating that linear regression tasks benefit less from increased parallelism and more from core availability per task, making them more task-intensive.

Configuration 3 ("Maximum Executors") demonstrates higher execution times for both pre-processing and linear regression due to excessive task scheduling overhead caused by over-distributing resources. Configuration 4 ("Balanced Parallelism") achieves a balance between resource utilization and task execution, with execution times close to Configuration 2 (1.38 minutes for pre-processing and 0.79 minutes for linear regression). The slightly higher execution time compared to Configuration 2 can be attributed to reduced task-level parallelism. Configuration 2's performance as the fastest for pre-processing highlights its suitability for highly parallelizable tasks. However, for linear regression, the equivalence of Configurations 1 and 2 in execution times suggests that core availability per task is more critical for computational efficiency in task-intensive workloads.

CNN and LSTM Model Training. Table 2 compares the execution times for CNN and LSTM training across three configurations specifically designed to address the requirements of Spark's TensorFlow distributor and MirroredStrategy.

Configuration 1 ("Three executors with four cores per executor and two cores per task") achieves the fastest execution times for both LSTM (11.60 minutes) and CNN (5.54 minutes). This configuration effectively balances resource allocation by reserving two cores

per task, ensuring that each task has sufficient computational power for training and synchronization. This combination of computational resources and efficient synchronization across three parallel jobs makes Configuration 1 particularly well-suited for complex deep-learning models.

In contrast, Configuration 2 ("Three executors with four cores per executor and one core per task") enables the maximum number of parallel jobs (nine), which theoretically should improve throughput. However, the reduced core allocation per task limits the computational resources available for training, leading to longer execution times (21.20 minutes for LSTM and 10.16 minutes for CNN). The single core per task in Configuration 2 appears insufficient to handle the intensive computation and synchronization demands of TensorFlow's MirroredStrategy, resulting in reduced performance despite greater parallelism.

Configuration 3 ("Six executors with two cores each and one core per task") compromises parallelism and computational power per task. While it achieves moderate execution times (15.40 minutes for LSTM and 10.96 minutes for CNN), it does not outperform Configuration 1. The reason is the insufficient computational power per task compared to Configuration 1, which dedicates more cores per task for intensive training and synchronization.

These results show that Configuration 1 is the optimal choice for training deep learning models like LSTM and CNN, as the reservation of two cores per task ensures that both training and barrier synchronization are efficiently handled.

Data Pre-processing Algorithm Comparison. Table 3 shows a negligible difference in execution times between groupByKey() and reduceByKey() algorithms for data pre-processing, with groupByKey() taking 0.54 minutes and reduceByKey() taking 0.53 minutes. This slight difference highlights that both algorithms perform well under the optimal Configuration 2. However, the slightly faster performance of reduceByKey() can be attributed to its memory efficiency. Unlike groupByKey(), which requires shuffling all grouped data to the same node, reduceByKey() performs aggregation locally on each node before shuffling, reducing network overhead. This advantage would be more significant if our dataset size were larger, making reduceByKey() the better choice for large-scale pre-processing.

4.3.2 Model Performance and Hyperparameter Tuning Discussion.

Linear Regression. Table 4 indicates that the linear regression model achieves an MAE of 2.66, an MSE of 11.41, and an RMSE of 3.38, with an accuracy of 23% within a margin of error of one degree. These results highlight the limitations of linear regression, which lacks the complexity needed to capture non-linear relationships and temporal dependencies in temperature data. While linear regression is efficient with certain limitations, its performance is significantly lower compared to LSTM and CNN models with predictions for shorter horizons. This supports its role as a baseline model, providing a reference point for evaluating more advanced approaches.

LSTM and CNN. Table 5 compares LSTM and CNN models across different forecast horizons and hyperparameter configurations. The LSTM model consistently outperforms CNN in terms of accuracy, particularly for one-day and seven-day forecasts. At a learning rate of 0.01 and batch size of 64, LSTM has an MAE of 1.95 and an accuracy of 32% for one-day forecasts, compared to CNN's MAE of 3.33 and accuracy of 19% at the same learning rate and batch size. CNN´s most optimal configuration in terms of metrics is 26% accuracy with a learning rate of 0.001 and a batch size of 64, which still falls short of the LSTM´s configuration with a 0.01 learning rate.

The superior performance of LSTM can be due to its capability to capture long-term temporal dependencies, which are critical for weather data. While effective for localized patterns, CNN struggles with long-term dependencies, leading to lower accuracy.

Hyperparameter tuning reveals that a learning rate of 0.01 generally improves performance for LSTM. However, smaller batch sizes, 16, result in longer training times, with marginal accuracy gains, as seen in CNN's 15-minute training time for one-day forecasts at a batch size of 16 compared to 5.54 minutes at a batch size of 64.

The LSTM model's robustness and its ability to handle temporal dependencies make it the preferred choice for this project. CNN, despite its limitations compared to LSTM, complements LSTM by focusing on localized patterns, offering additional insights.

As the prediction horizon increases, the LSTM and CNN models show declining accuracy, with metrics like MAE and RMSE worsening for 30-day predictions. Interestingly, linear regression performs better over longer horizons, as seen in the 30-day results, likely due to its simplicity and reduced risk of overfitting. The plots Appendix 7 show that CNN and LSTM predictions resemble those of linear regression as the horizon extends.

5 CONCLUSION

This project addressed the challenge of effectively predicting daily temperatures using a distributed data processing pipeline and machine learning models. By employing the Medallion Architecture, the pipeline provided a structured approach to data ingestion, preprocessing, and analysis, using the flexibility and scalability of Apache Spark.

We conducted experiments with multiple configurations to ensure optimal performance for both pre-processing and model training. Linear regression was a baseline model, showing its computational efficiency but limited accuracy for shorter prediction horizons. In contrast, the LSTM model emerged as the best performer, excelling at capturing temporal dependencies and achieving the highest accuracy across most metrics.

As prediction horizons extended, the LSTM and CNN models showed increasing similarity to linear regression, reflecting the growing uncertainty over time. These findings highlight the importance of model selection and configuration tuning in building robust and scalable forecasting systems.

5.1 Future Work

- Larger Dataset: Using a larger dataset could potentially result in easier observing and monitoring of differences in configurations made during this project's optimization phase.
- Other Models: Exploring other models could provide better predictive capabilities and insights into the problem space.

- Horizon-Specific Models: Developing separate models tailored to each prediction horizon (e.g., 1-day, 7-day, 30-day) could optimize accuracy for specific forecasting needs
- Explore different data augmentation techniques: While this project utilized transformations like sine and cosine to capture seasonality, future work could explore advanced data augmentation techniques such as noise injection, synthetic data generation, or time-series resampling.

6 ACKNOWLEDGMENTS

- In this project, generative AI and Grammarly have been used to help with reformulations and enhance the readability of the thesis and not as a source of information.
- ChatGPT and Copilot have been used during the development to help with code suggestions and debugging.

REFERENCES

[1] [n. d.]. Apache Hadoop. https://hadoop.apache.org/
[2] [n. d.]. Configuration - Spark 3.5.1 Documentation. https://spark.apache.org/docs/3.5.1/configuration.html
[3] [n. d.]. ecosystem/spark/spark-tensorflow-distributor at master · tensorflow/ecosystem. https://github.com/tensorflow/ecosystem/tree/master/spark/spark-tensorflow-distributor
[4] [n. d.]. Frost API. https://frost.met.no/index.html
[5] [n. d.]. Mean squared error (MSE) | Definition, Formula, Interpretation, & Facts | Britannica. https://www.britannica.com/science/mean-squared-error
[6] [n. d.]. Root Mean Squared Error (RMSE) | SAP Help Portal. https://help.sap.com/docs/SAP_PREDICTIVE_ANALYTICS/41d1a6d4e7574e32b815f1cc87c00f42/5e5198fd4afe4ae5b48fefe0d3161810.html
[7] [n. d.]. tf.distribute.MirroredStrategy | TensorFlow v2.16.1. https://www.tensorflow.org/api_docs/python/tf/distribute/MirroredStrategy
[8] [n. d.]. Weatherstack - Real-Time World Weather REST API. https://weatherstack.com/
[9] [n. d.]. What is Spark? - Introduction to Apache Spark and Analytics - AWS. https://aws.amazon.com/what-is/apache-spark/
[10] avanwyk . [n. d.]. Encoding Cyclical Features for Deep Learning. https://www.avanwyk.com/encoding-cyclical-features-for-deep-learning/
[11] M. Waqar Ahmed. [n. d.]. Understanding Mean Absolute Error (MAE) in Regression: A Practical Guide. https://medium.com/@m.waqar.ahmed/understanding-mean-absolute-error-mae-in-regression-a-practical-guide-26e80ebb97df
[12] Zach Bobbitt. [n. d.]. How to Perform Linear Regression in PySpark (With Example). https://www.statology.org/pyspark-linear-regression/
[13] Sigurd Heggemoen and Alan Lici. [n. d.]. Sihe12/Dat535_project. https://github.com/Sihe12/Dat535_project original-date: 2024-10-20T21:43:06Z.
[14] KAVITA. [n. d.]. Linear Regression: A Comprehensive Guide. https://www.analyticsvidhya.com/blog/2021/10/everything-you-need-to-know-about-linear-regression/
[15] Thejas Kiran. [n. d.]. Unlocking the Potential of Convolutional Neural Networks (CNNs) in Time Series Forecasting. https://thejaskiran99.medium.com/unlocking-the-potential-of-convolutional-neural-networks-cnns-in-time-series-forecast
[16] Christopher Olah. [n. d.]. Understanding LSTM Networks – colah's blog. https://colah.github.io/posts/2015-08-Understanding-LSTMs/
[17] Dani Pálma. [n. d.]. Apache Parquet for Data Engineers: Optimized Data Storage. https://estuary.dev/apache-parquet-for-data-engineers
[18] rimmalapudi. [n. d.]. Tune Spark Executor Number, Cores, and Memory. https://sparkbyexamples.com/spark/spark-tune-executor-number-cores-and-memory/
[19] Sebastien Simon. [n. d.]. Forecasting Weather Patterns with LSTM: A Python Guide Without Dates. https://medium.com/@sebastienwebdev/forecasting-weather-patterns-with-lstm-a-python-guide-without-dates-433f0356136c

7 APPENDIX

7.1 Linear Regression

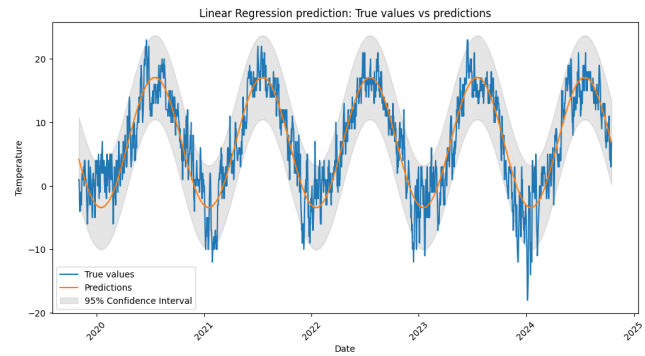

Figure 2: Linear regression plot of the actual values versus the predicted values, along with a 95% confidence interval.

7.2 Long Short-Term Memory

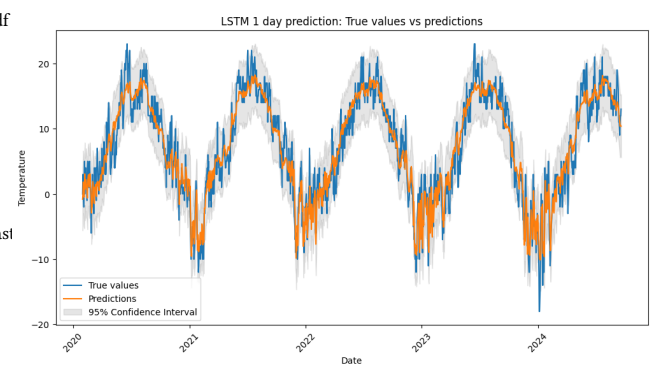

Figure 3: LSTM one day prediction plot of the actual values versus the predicted values, along with a 95% confidence interval using a learning rate of 0.01 and a batch size of 64 with configuration 1.

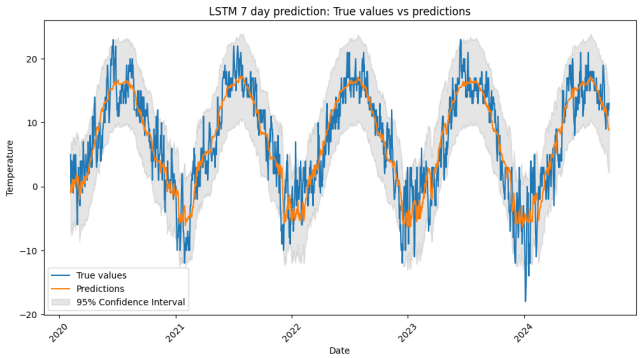

Figure 4: LSTM seven day prediction plot of the actual values versus the predicted values, along with a 95% confidence interval using a learning rate of 0.01 and a batch size of 64 with configuration 1.

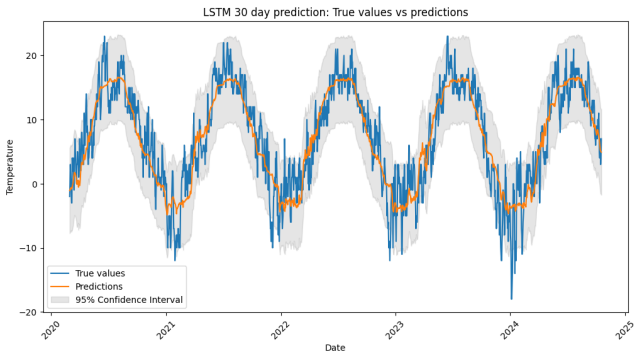

Figure 5: LSTM 30 day prediction plot of the actual values versus the predicted values, along with a 95% confidence interval using a learning rate of 0.01 and a batch size of 64 with configuration 1.

7.3 Convolutional Neural Network

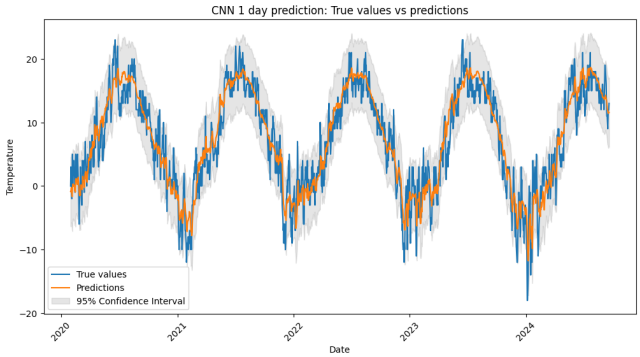

Figure 6: CNN one day prediction plot of the actual values versus the predicted values, along with a 95% confidence interval using a learning rate of 0.001 and a batch size of 64 with configuration 1.

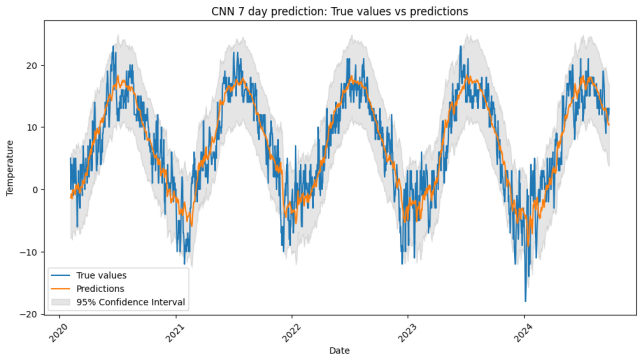

Figure 7: CNN seven day prediction plot of the actual values versus the predicted values, along with a 95% confidence interval using a learning rate of 0.001 and a batch size of 64 with configuration 1.

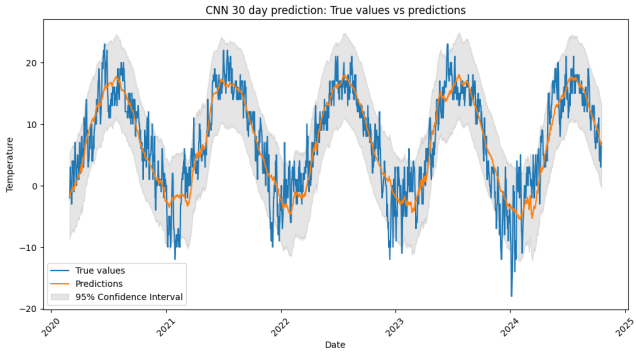

Figure 8: CNN 30 day prediction plot of the actual values versus the predicted values, along with a 95% confidence interval using a learning rate of 0.001 and a batch size of 64 with configuration 1.